Project Management

Project Management Artifacts: GraphOCR Deterministic Trust Layer

Author: Mohamed Hussein

Date: March 30, 2026

Status: Final for Review

This is a small breif not a complete RFC. It misses Jria cards

---

1. Request for Comments (RFC)

1.1 Objective & Problem Statement

The global health industry is burdened by an analog legacy, where 90% of health history is handwritten, multi-lingual, and physically archived on paper. Our objective is to process 100k complex insurance claims daily. These documents are multilingual (Arabic/English) scans of handwritten or illegible doctor notes, and receipts with overlapping stamps.

Current off-the-shelf AI pipelines (Standard OCR + RAG) fail at the institutional level due to two fatal errors:

- **Serialization Gore (Spatial-Blindness):** Normal OCR does not detect the position of each word. It reads horizontally across columns, mixing different patients' data or merging prices with dates, creating a meaningless soup of text.
- **Contextual Hallucination:** Standard RAG systems do not "see" the paper. They retrieve the most semantically similar policy (e.g., a 2025 standard plan) even if the handwritten claim refers to an obsolete 2018 rider, causing the AI to hallucinate a denial.

1.2 The Deterministic Trust Layer Architecture

To maintain 98% accuracy at scale, we require a Deterministic Trust Layer that grounds the AI in the reality of the document.

- **Layer 1: Spatial-Aware Foundation:** We must detect the region of words and express it by bounding boxes. We will use two OCR engines (Paddle and Surya) and merge their outputs. The system will preprocess data to handle rotation, lights, and saturation , and classify if the text structure is handwritten or printed. This enforces a consistent "Semantic Spatial" metadata schema across the system.

- **Layer 2: Hybrid Graph-OCR Validation (Self-Healing):** We will use a Neo4j Knowledge Graph to detect "Logical Impossibilities" in the OCR output, such as a 1940 birthdate on a 2026 policy, or drug dosages exceeding clinical limits. When the graph identifies a logical conflict, a Back-Propagation mechanism automatically triggers a targeted VLM Re-Scan of the specific source coordinates without restarting the entire 100k-doc batch. This creates a robust, Self-Healing pipeline.
- **Layer 3: Tiered Inference:** A "Traffic Controller" agent assigns an "Uncertainty Score" to every document. This threshold determines which 10% of documents require the expensive VLM-Consensus treatment, while the remaining 90% pass through standard rails.
- **Sovereignty & Security:** The architecture assumes the use of Federated systems that respect sovereign data constraints.

---

2. Task Management & Decomposition

The architecture is divided into three main parts. To execute this massive pipeline efficiently, the workload is distributed among 3 Developers and 1 Team Lead (TL).

2.1 Developer 1 (Data & Spatial Layer)

- **Ingestion & Preprocessing:** Build the preprocessing pipeline for data, handling rotation, lights, and saturation.
- **Dual-Engine OCR:** Implement the use of two engines (Paddle and Surya) and merge their outputs.
- **FTI Foundation:** Handle the data restoration and setup for the FTI (Feature, Training, Inference) architecture.

2.2 Developer 2 (Routing & Agent System)

- **Traffic Controller:** Build the routing logic from the router to the standard agent system.
- **Scoring Mechanism:** Implement the logic to assign a score for each document.

2.3 Developer 3 (Graph, RAG & VLM)

- **Advanced Routing:** Build the connection from the router to the RAG system and Neo4j.
- **VLM Integration:** Implement the real VLM part for failed or highly uncertain documents.
- **Deterministic Logic:** Configure the Neo4j Graph Database.

2.4 Team Lead (Architecture & Governance)

- **Integration:** Manage the connection between layers and divide each layer's tasks.
- **Agentic Supervision:** Design an "Agentic Supervisor" to watch the Junior use DSPy. The TL/Supervisor will step in to explain the "math of textual gradients" when the prompt optimization starts to hallucinate or drift.

---

3. Release Strategy & Operational Plan

Deploying this system without disrupting daily operations requires a highly controlled, phased approach monitored by Langfuse at scale to track "Accuracy Decay" in real-time.

3.1 Phased Rollout

- **Phase 1: Pre-Release (Shadow Mode):** Deploy the system to run passively. We will collect the data output of the router to evaluate performance.
- **Phase 2: Fine-Tuning Prep:** Prepare the collected data to fine-tune the VLM.
- **Phase 3: Pre-Release with Users:** Roll out the system with user feedback mechanisms enabled.

3.2 Automated Circuit Breaker

If accuracy dips to 95.5% at 2:00 PM, an automated "Circuit Breaker" strategy will trigger. This will halt standard processing and temporarily route traffic to human review or higher-tier models until the root cause is resolved, preventing cascading systemic failures.

---

4. Future Architecture Enhancements

To guarantee long-term stability and continuous improvement, the following architectural enhancements are scheduled post-launch:

- **Dynamic Error Feedback:** Implement mechanisms for dynamic error feedback to continuously update the system's core logic.
- **Full FTI Architecture:** Fully realize the FTI (Feature, Training, Inference) architecture. We must ensure the features part is independent rather than connected.

- **ETL Concept Integration:** Introduce standard ETL (Extract, Transform, Load) concepts for robust data pipeline management across sovereign regions.